15
Combining Computer Vision and NLP Techniques

In the previous chapter, we learned about applications that combine reinforcement learning and computer vision. In this chapter, we will switch gears and learn about how a **convolutional neural network** (**CNN**) can be used in conjunction with algorithms in the broad family of **transformers**, which are heavily used (as of the time of writing this book) in **natural language processing** (**NLP**) to develop solutions that leverage both computer vision and NLP.

To understand combining CNNs and transformers, we will first learn how **vision transformers** (**ViTs**) work and how they help in performing image classification. After that, we will learn about leveraging transformers to perform the transcription of handwritten images using **Transformer optical character recognition** (**TrOCR**). Next, we will learn about combining transformers and OCR to perform question answering on document images using a technique named **LayoutLM**. Finally, we will learn about performing visual question answering using a transformer architecture named **Bootstrapping Language Image Pre-training** (**BLIP2**).

By the end of this chapter, you will have learned about the following topics:

- Implementing ViTs for image classification
- Implementing LayoutLM for document question answering
- Transcribing handwritten images
- Visual question answering using BLIP2

You are advised to go through the supplemental chapter on training with minimal data points to get familiarity with word embeddings, available in the `Extra chapters from first edition` folder on GitHub.

The code in this chapter is available in the `Chapter15` folder of this book's GitHub repository at https://bit.ly/mcvp-2e.

As the field evolves, we will periodically add valuable supplements to the GitHub repository. Do check the

`supplementary_sections` folder within each chapter's directory for new and useful content.

Introducing transformers

Before we learn about ViTs, let us understand transformers from an NLP perspective. A transformer helps in generating a representation (word/vector embedding) that best describes the word given its context (surrounding words). Some of the major limitations of **recurrent neural networks (RNNs)** and **long short-term memory (LSTM)** architecture (detailed information about which is provided in the associated GitHub repository) are:

- A word embedding corresponding to a word is not dependent on the context in which the word appears (the word *apple* will have the same embedding irrespective of whether the context is about the fruit or the company).
- Hidden state calculation during training happens sequentially (a word's hidden state is dependent on the previous word's hidden state and thus can only be calculated after the previous hidden state is calculated), resulting in a considerable time taken to process text.

Transformers address these two major limitations, which results in:

- The ability to pre-train transformers on a large corpus of data
- Fine-tuning transformers to a variety of downstream tasks (including leveraging them for vision tasks)
- Leveraging the intermediate states/hidden states in a variety of architectures – encoder-only, decoder-only, or encoder-decoder architectures (more on encoders and decoders in the following section)
- The ability to train transformer outputs in parallel when compared to sequentially in RNN

With the advantages of transformers in place, let us understand how they work.

Basics of transformers

To understand transformers, let us go through a scenario of machine translation – where the source language is English and the target language is French.

A transformer architecture can be illustrated as follows:

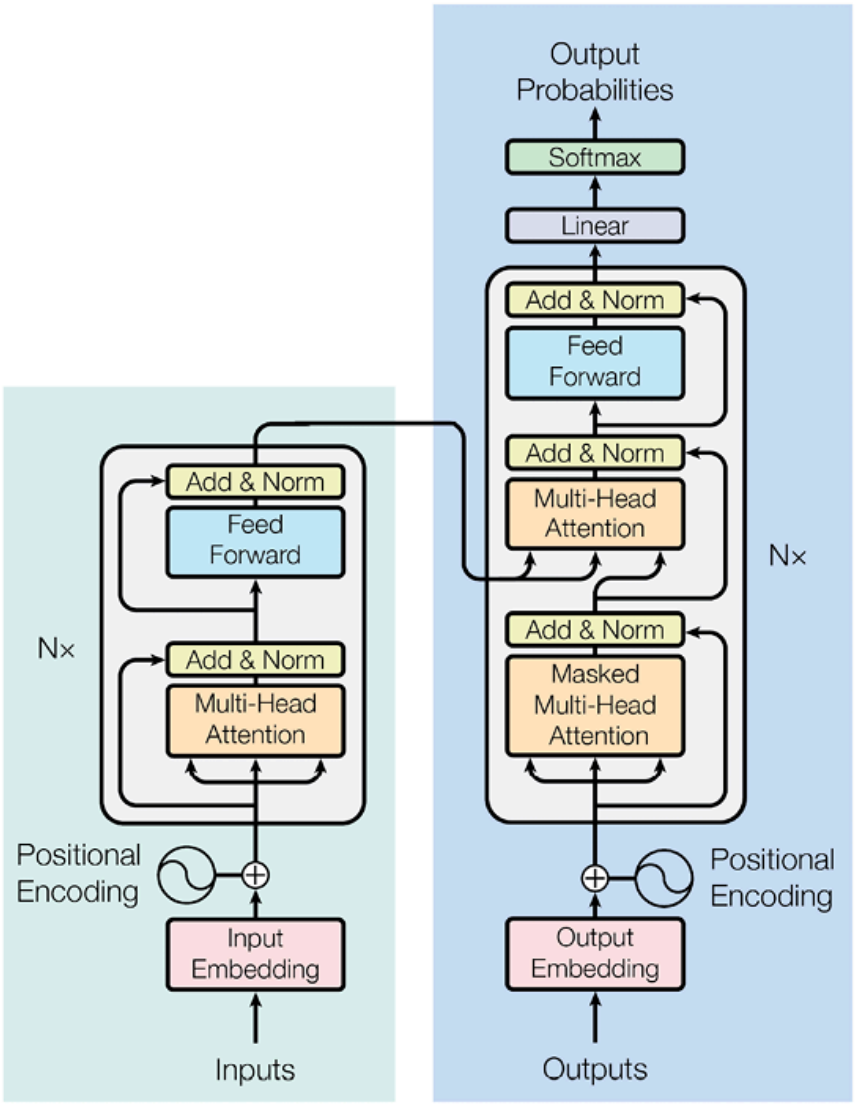

Figure 15.1: Transformer architecture

(Source: https://arxiv.org/pdf/1706.03762)

In the above architecture, the left-hand-side block is the encoder and the right-hand-side one is the decoder. Let us first understand the encoder block.

Encoder block

The first step is to fetch the tokens corresponding to the input sentence in English. In traditional language modeling, we assign an "unknown" token to rare words and fetch the embeddings corresponding to the remaining (frequent) words. However, during the tokenization process of transformer architecture, we perform byte pair encoding (tokenization) in such a way that we break individual words (for example, the word `anand` could be broken into `###an`, `###and`, while a frequent word like `computer` would remain as is). This way, we would not have any unknown

words. Additionally, each token would then have an embedding associated with it.

Thus, we leverage tokenization to obtain the input word embeddings.

Let's now learn about the **self-attention** module, which is at the heart of a transformer. It takes three two-dimensional matrices – called **query** (**Q**), **key** (**K**), and **value** (**V**) matrices – as input. The matrices can have very large embedding sizes (as they would contain vocabulary x embedding size number of values), so they are split up into smaller components first (*Step 1* in the following diagram), before running through the scaled dot-product attention (*Step 2* in the following diagram):

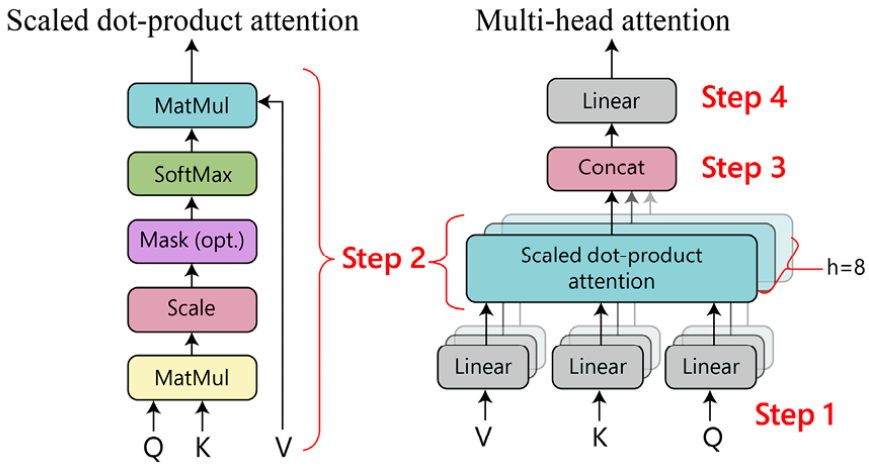

Figure 15.2: Workflow of scaled-dot product attention and multi-head attention

Let's understand how self-attention works. Let's imagine a hypothetical scenario where the sequence length of input is 3 tokens – i.e., we have three word/token embeddings (*W1*, *W2*, and *W3*) as input. Say each embedding is of size 512 values. The following steps can be performed:

1. Each of these embeddings is individually converted into three additional vectors, which are the `Q`, `K`, and `V` vectors corresponding to each input. In the following image, `512` is the embedding dimension of the Q, K, and V vectors and `3` is the sequence length (3 words/tokens):

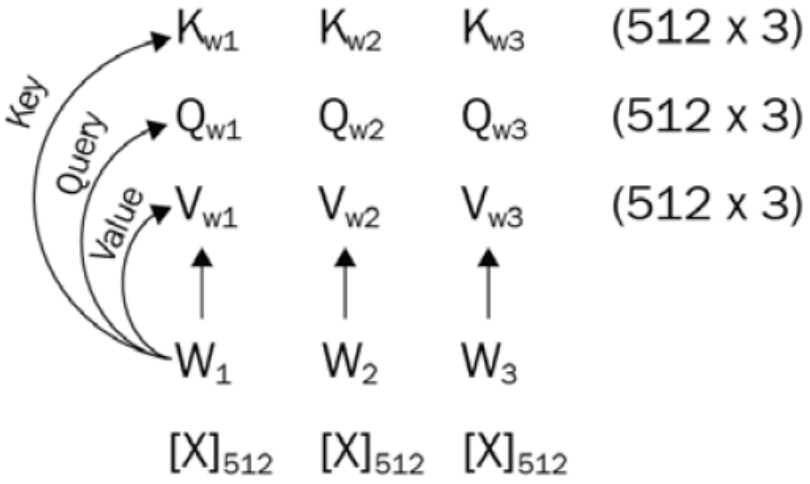

Figure 15.3: Initializing Q, K, and V vectors

2. We use a multi-head approach where we split each of these vectors into smaller "heads" (eight in this example), with eight sets of vectors (of size 64 x 3) for each key, query, and value tensor. Here, `64` is obtained by dividing 512 (embedding size) by 8 (number of heads), and `3` is the sequence length:

$$K_{w11} \quad K_{w21} \quad K_{w31} \quad (64 \times 3) = K_w$$
$$Q_{w11} \quad Q_{w21} \quad Q_{w31} \quad (64 \times 3) = Q_w$$
$$V_{w11} \quad V_{w21} \quad V_{w31} \quad (64 \times 3) = V_w$$

Figure 15.4: Q, K, and V values of each head

Note that there will be eight sets of tensors of key, query, and value as there are eight heads. Furthermore, each head could learn about different aspects of a word.

3. In each head, we first perform matrix multiplication between the key transpose and query matrices. This way, we end up with a 3 x 3 matrix. Divide the resulting matrix by the square root of the number of dimensions of vectors (`d = 64` in this case). Pass it through softmax activation. Now, we have a matrix showing how important each word is in relation to every other word:

$$\underset{(3\times 64)\ (64\times 3)}{\underset{(3\times 3)}{K_w^T Q_w}} \rightarrow \left(\frac{K_w^T Q_w}{\sqrt{d_k}}\right) \rightarrow Softmax\left(\frac{K_w^T Q_w}{\sqrt{d_k}}\right)$$

Figure 15.5: Operations on Q and K vectors

4. Finally, we perform matrix multiplication of the preceding tensor output with the value tensor to get the output of our self-attention operation:

$$V_w \quad Softmax\left(\frac{K_w^T Q_w}{\sqrt{d_k}}\right) \rightarrow \quad Z$$
$$(64 \times 3) \quad (3 \times 3) \quad (64 \times 3)$$

Figure 15.6: Self-attention calculation

Formally, this scaled-dot product attention calculation can be written as:

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

In the preceding equation, *d* refers to the dimension of vector (64 in this case) and k represents the index of head.

A step-by-step calculation on a sample input and randomly initialized Q, K, and V weight matrices is provided in the associated GitHub repository as `self-attention.ipynb` in the `Chapter15` folder at https://bit.ly/mcvp-2e.

5. We then combine the eight outputs of this step using the concat layer (*Step 3* in *Figure 15.2*), and end up with a single tensor of size 512 x 3. As there are eight heads (i.e., eight Q, K, and V matrices), the layer is also called **multi-head self-attention** (source: *Attention Is All You Need*, https://arxiv.org/pdf/1706.03762.pdf).

For our example in computer vision, when searching for an object such as a horse, the query would contain information to search for an object that is large in dimension and is brown, black, or white in general. The softmax output of scaled dot-product attention will reflect those parts of the key matrix that contain this color (brown, black, white, and so on) in the image. Thus, the values output from the self-attention layer will have those parts of the image that are roughly of the desired color and are present in the values matrix.

6. We then pass the output of multi-head attention through a residual block, in which we first add the inputs and output of multi-head attention, and then perform normalization of this final output.

7. Then, we pass the output through a linear network to get the output, which has similar dimensions as that of the input.

We use the self-attention block several times (`Nx` in *Figure 15.1*).

Decoder block

While the decoder block is very similar to the encoder block, there are two additions to the architecture:

- Masked multi-head attention
- Cross-attention

While multi-head attention works in a manner similar to the encoder block, we mask the tokens in future timesteps while calculating the attention of a token at a given timestep. This way, we are building the network so that it does not peek into future tokens. We mask future tokens by adding a mask/identity matrix, which ensures that future tokens are not considered during attention calculation.

The key and value matrices of the encoder are used as the key and query inputs for the cross-head attention of the decoding half, while the value input is learned by the neural network, independent of the encoding half. We call it cross-attention as key and value matrices are fetched from the encoder layer while the query is fetched from the decoder layer.

Finally, even though this is a sequence of inputs, there's no sign of which token (word) is first and which is next. Positional encodings are learnable embeddings that we add to each input as a function of its position in the sequence. This is done so that the network understands which word embedding is first in the sequence, which is second, and so on.

The way to create a transformer network in PyTorch is very simple. There is a built-in transformer block that you can create, like so:

```
from torch import nn
transformer = nn.Transformer(hidden_dim, nheads, \
                    num_encoder_layers, num_decoder_layers)
```

Here, `hidden_dim` is the size of the embeddings, `nheads` is the number of heads in the multi-head self-attention, and `num_encoder_layers` and `num_decoder_layers` are the number of encoding and decoding blocks in the network, respectively.

The working details of transformers and their implementation from scratch is provided in the `Transformers_from_scratch.ipynb` file within the `Chapter15` folder of the GitHub repository at https://bit.ly/mcvp-2e.

Now that we know how transformers work, in the next section, let us learn how ViTs work.

How ViTs work

A ViT can be easily understood with the following diagram:

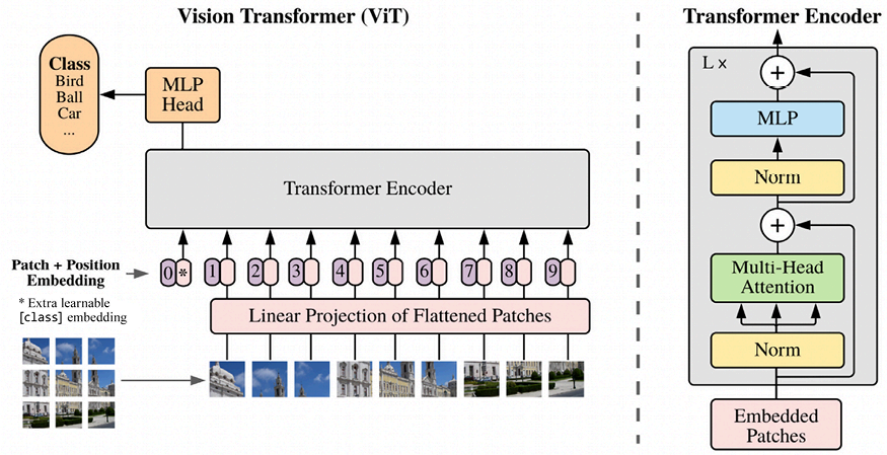

Figure 15.7: ViT architecture (source: https://arxiv.org/pdf/2010.11929)

Let's look at the workflow that is being followed in the preceding figure:

1. We are taking an image and extracting 9 (3 x 3) patches from it. For example, let us assume the original image is of size 300 x 300 and each of the 9 patches would be 100 x 100 in shape.
2. Next, we take each of the patches, flatten them, and pass them through a linear projection. This exercise is like extracting word embeddings corresponding to a word (token).
3. Next, we add the positional embedding corresponding to the patch. We add a positional embedding as we need to preserve the information of the location of the patch in original image. In addition, we are also initializing a class token that would be helpful in the final classification of image.
4. Now, all the embeddings are passed through the transformer encoder. In a transformer, these embeddings pass through a sequence of nor-

malization, multi-head attention, and a skip connection with a linear head (MLP stands for multi-layer perceptron).

5. Now, we have output embeddings corresponding to each patch. We now attach the final head, depending on the problem we are trying to solve. In the case of image classification, we would attach a linear layer only for the **classification (CLS)** token. The outputs of remaining patches can be used as a feature extractor for downstream tasks like object recognition or image captioning.

Now that we understand how ViTs work, we will go ahead and implement transformers on a dataset.

Implementing ViTs

We will implement ViTs on the cats vs dogs dataset that we leveraged in *Chapters 4* and *5*:

The following code is available in the `ViT_Image_classification.ipynb` file in the `Chapter15` folder of this book's GitHub repository at `https://bit.ly/mcvp-2e`.

1. Install and import the required packages:

```
%pip install -U torch-snippets transformers kaggle
from torch_snippets import *
from transformers import ViTModel, ViTConfig
from torch.optim import Adam
model_checkpoint = 'google/vit-base-patch16-224-in21k'
```

Note that we will be leveraging the pre-trained ViT model (checkpoint location provided above).

2. Import the dataset:

```
%%writefile kaggle.json
{"username":"xx", "key":"xx"}
!mkdir -p ~/.kaggle
!cp kaggle.json ~/.kaggle/
!chmod 600 /root/.kaggle/kaggle.json
!kaggle datasets download -d tongpython/cat-and-dog
!unzip cat-and-dog.zip
```

3. Specify the training data location:

```
train_data_dir = 'training_set/training_set'
test_data_dir = 'test_set/test_set'
```

4. Specify the dataset class as we did in *Chapters 4* and *5*:

```
class CatsDogs(Dataset):
    def __init__(self, folder):
        cats = glob(folder+'/cats/*.jpg')
        dogs = glob(folder+'/dogs/*.jpg')
        self.fpaths = cats[:500] + dogs[:500]
        self.normalize = transforms.Normalize(mean=[0.485, 0.456, 0.406],
                                              std=[0.229, 0.224, 0.225])
        from random import shuffle, seed; seed(10); shuffle(self.fpaths)
        self.targets = [fpath.split('/')[-1].startswith('dog') \
                                    for fpath in self.fpaths]
    def __len__(self): return len(self.fpaths)
    def __getitem__(self, ix):
        f = self.fpaths[ix]
        target = self.targets[ix]
        im = (cv2.imread(f)[:,:,::-1])
        im = cv2.resize(im, (224,224))
        im = torch.tensor(im/255)
        im = im.permute(2,0,1)
        im = self.normalize(im)
        return im.float().to(device), \
                  torch.tensor([target]).float().to(device)
```

5. Define the ViT model architecture:

```
class ViT(nn.Module):
  def __init__(self, config=ViTConfig(), num_labels=1,
               model_checkpoint='google/vit-base-patch16-224-in21k'):
        super(ViT, self).__init__()
        self.vit = ViTModel.from_pretrained(model_checkpoint, \
                                    add_pooling_layer=False)
        self.classifier1 = (nn.Linear(config.hidden_size, 128))
        self.classifier2 = (nn.Linear(128, num_labels))
        self.classifier = nn.Sequential(
                                      self.classifier1,
                                      nn.ReLU(),
                                      self.classifier2)
        for param in self.vit.parameters():
            param.requires_grad = False
  def forward(self, x):
    x = self.vit(x)['last_hidden_state']
    # Use the embedding of [CLS] token
    output = self.classifier(x[:, 0, :])
    output = torch.sigmoid(output)
    return output
```

In the preceding architecture, we are fetching the features corresponding to each of the patches, fetching the feature of the first one (CLS token) and then passing it through a sigmoid layer because we want to classify it as one of the possible classes.

6. Define the model, loss function, and optimizer:

```
model = ViT().to('cuda')
loss_fn = nn.BCELoss()
optimizer = torch.optim.Adam(model.parameters(), lr= 1e-3)
```

7. Define the functions to perform training, calculate accuracy, and fetch data:

```
def train_batch(x, y, model, opt, loss_fn):
    model.train()
    prediction = model(x)
    batch_loss = loss_fn(prediction, y)
    batch_loss.backward()
    optimizer.step()
    optimizer.zero_grad()
    return batch_loss.item()
@torch.no_grad()
def accuracy(x, y, model):
    model.eval()
    prediction = model(x)
    is_correct = (prediction > 0.5) == y
    return is_correct.cpu().numpy().tolist()
def get_data():
    train = CatsDogs(train_data_dir)
    trn_dl = DataLoader(train, batch_size=32, shuffle=True,
                                            drop_last = True)
    val = CatsDogs(test_data_dir)
    val_dl = DataLoader(val, batch_size=32, shuffle=True,
                                            drop_last = True)
    return trn_dl, val_dl
trn_dl, val_dl = get_data()
```

8. Train the model over increasing epochs:

```
n_epochs = 5
report = Report(n_epochs)
for epoch in range(n_epochs):
    train_epoch_losses, train_epoch_accuracies = [], []
    val_epoch_accuracies = []
    n = len(trn_dl)
    for ix, batch in enumerate(iter(trn_dl)):
        x, y = batch
```

```
            batch_loss = train_batch(x, y, model, optimizer, loss_fn)
            is_correct = accuracy(x, y, model)
            report.record(epoch+(ix+1)/n, trn_loss=batch_loss,
                                  trn_acc=np.mean(is_correct), end='\r')
        n = len(val_dl)
        for ix, batch in enumerate(iter(val_dl)):
            x, y = batch
            val_is_correct = accuracy(x, y, model)
            report.record(epoch+(ix+1)/n,
                          val_acc=np.mean(val_is_correct), end='\r')
        report.report_avgs(epoch+1)
```

9. Training and validation accuracy are as follows:

```
report.plot(['trn_loss'], sz=3, figsize=(5,3))
report.plot_epochs(['acc','trn_acc'], figsize=(5,3))
```

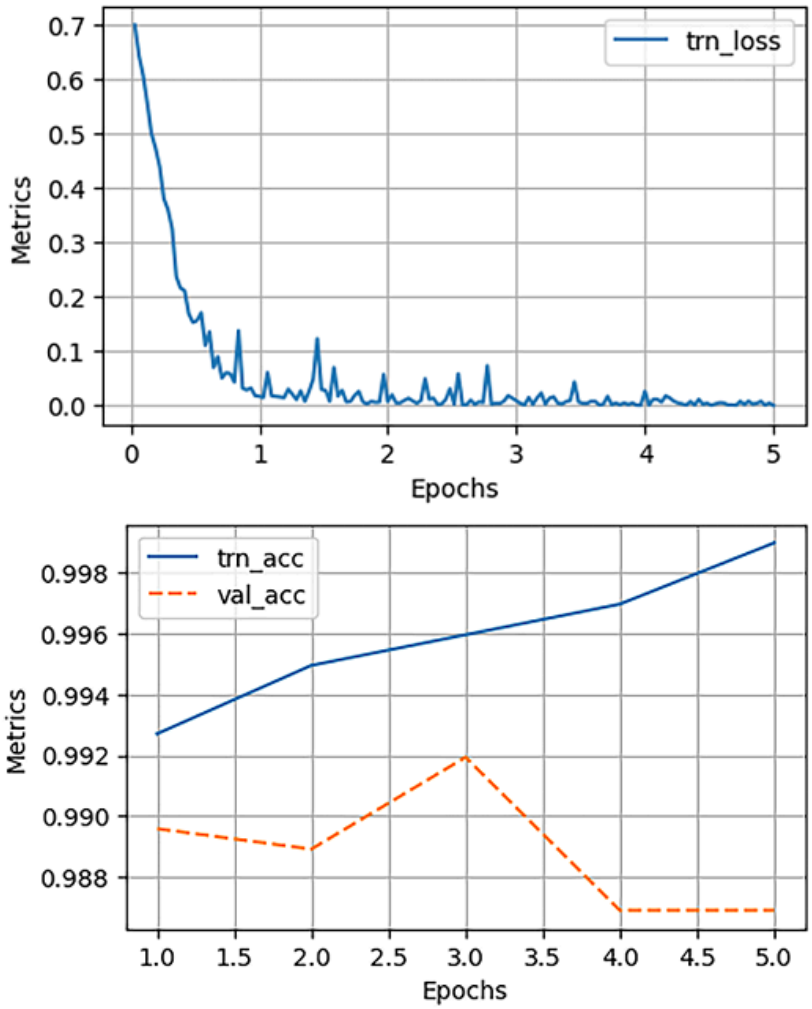

Figure 15.8: Training loss and accuracy over increasing epochs

Note that the accuracy is similar to the accuracy that we saw with VGG and ResNet in *Chapter 5*.

In this section, we learned about leveraging an encoder-only transformer to perform image classification. In the next section, we will learn about leveraging encoder-decoder architecture-based transformers to transcribe images containing handwritten words.

Transcribing handwritten images

Imagine a scenario where you must extract information from a scanned document (extracting keys and values from a picture of an ID card or a picture of a manually filled-in form). You'll have to extract (transcribe) text from the image. This problem gets tricky due to variety in the following:

- Handwriting
- Quality of the scan/picture
- Lighting conditions

In this section, we will learn about the technique to transcribe handwritten images.

Let us first understand how an encoder-decoder architecture of a transformer can be applied to transcribe a handwritten image.

Handwriting transcription workflow

We will leverage the TrOCR architecture (source: https://arxiv.org/abs/2109.10282) to transcribe handwritten information.

The following diagram shows the workflow that is followed:

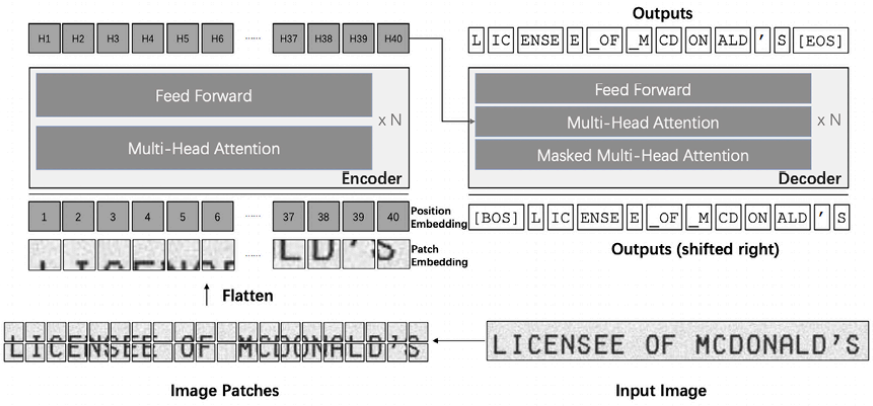

Figure 15.9: TrOCR workflow

As shown in the preceding picture, the workflow is as follows:

1. We take an input and resize it to a fixed height and width.
2. Then, we divide the image into a set of patches.
3. We then flatten the patches and fetch embeddings corresponding to each patch.
4. We combine the patch embeddings with position embeddings and pass them through an encoder.
5. The key and value vectors of the encoder are fed into the cross-attention of the decoder to fetch the outputs in the final layer.

The tokenizer and the model that we will use to train the model will leverage the `trocr-base-handwritten` model released by Microsoft. Let us go ahead and code up handwriting recognition in the next section.

Handwriting transcription in code

The following steps can be followed for handwriting transcription:

This code is available as `TrOCR_fine_tuning.ipynb` in the `Chapter15` folder of this book's GitHub repository at https://bit.ly/mcvp-2e.

1. Download and import the dataset of images:

```
!wget https://www.dropbox.com/s/l2ul3upj7dkv4ou/synthetic-data.zip
!unzip -qq synthetic-data.zip
```

In the preceding code, we have downloaded the dataset in which the images are provided; the filename of the image contains the ground truth of transcription corresponding to that image.

A sample from the images that were downloaded is as follows:

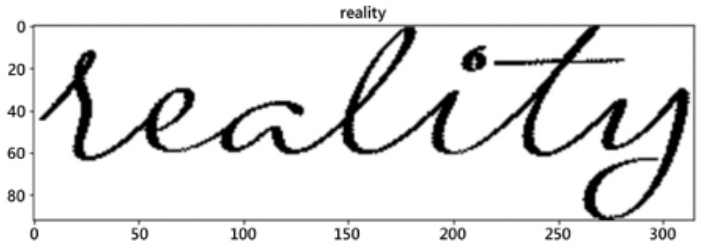

Figure 15.10: Sample image along with ground truth (as title)

2. Install the required packages and import them:

```
!pip install torch_snippets torch_summary editdistance jiwer accelerate
from torch_snippets import *
from torchsummary import summary
import editdistance
```

3. Specify the location of the images and the function to fetch the ground truth from the images:

```
device = 'cuda' if torch.cuda.is_available() else 'cpu'
fname2label = lambda fname: stem(fname).split('@')[0]
images = Glob('synthetic-data/*')
```

Note that we are creating the `fname2label` function as the ground truth of an image is available after the `@` symbol in the filename. A sample of the filenames is as follows:

```
['synthetic-data/already@dPkHBT.png',
 'synthetic-data/bring@OzNTFr.png',
 'synthetic-data/few@EhVOvU.png',
 'synthetic-data/research@cgo7rI.png',
 'synthetic-data/fast@bQ8Wkm.png']
```

Figure 15.11: Sample filenames

4. Fetch 5,000 samples (images and their corresponding labels) from the 25K image dataset for faster training:

```
images_list = []
labels_list = []
for image in images:
  images_list.append(str(image).split('/')[-1])
  labels_list.append(fname2label(image))
df = pd.DataFrame([images_list[:5000], labels_list[:5000]]).T
df.columns = ['file_name', 'text']
```

5. Specify the training and test DataFrames:

```
from sklearn.model_selection import train_test_split
train_df, test_df = train_test_split(df, test_size=0.1)
train_df.reset_index(drop=True, inplace=True)
test_df.reset_index(drop=True, inplace=True)
```

6. Define the `Dataset` class:

```
class IAMDataset(Dataset):
    def __init__(self, root_dir, df, processor, max_target_length=128):
        self.root_dir = root_dir
        self.df = df
        self.processor = processor
        self.max_target_length = max_target_length
    def __len__(self):
        return len(self.df)
    def __getitem__(self, idx):
        # get file name + text
        file_name = self.df['file_name'][idx]
        text = self.df['text'][idx]
        # prepare image (i.e. resize + normalize)
        image = Image.open(self.root_dir + file_name).convert("RGB")
        pixel_values = self.processor(image,
                                      return_tensors="pt").pixel_values
        # add labels (input_ids) by encoding the text
        labels = self.processor.tokenizer(text,
                                          padding="max_length",
                                max_length=self.max_target_length).input_ids
        # important: make sure that PAD tokens are ignored by the loss function
        labels = [label if label != self.processor.tokenizer.pad_token_id \
                                             else -100 for label in labels]
        encoding = {"pixel_values": pixel_values.squeeze(), \
                    "labels": torch.tensor(labels)}
        return encoding
```

In the preceding code, we are fetching the image, passing through a processor to fetch the pixel values. Furthermore, we are passing the label through the tokenizer of the model to fetch the tokens of labels.

7. Define `TrOCRProcessor`:

```
from transformers import TrOCRProcessor
processor = TrOCRProcessor.from_pretrained("microsoft/trocr-base-handwritten")
```

In the preceding code, we are defining the processor that preprocesses the images and performs the tokenization of labels.

8. Define the training and evaluation datasets:

```
train_dataset = IAMDataset(root_dir='/content/synthetic-data/', \
                           df=train_df, processor=processor)
eval_dataset = IAMDataset(root_dir='/content/synthetic-data/',
                           df=test_df, processor=processor)
```

9. Define the `TrOCR` model:

```
from transformers import VisionEncoderDecoderModel
model = VisionEncoderDecoderModel.from_pretrained("microsoft/trocr-base-
                                                    stage1")
```

10. Define the model configuration parameters and training arguments:

```
# set special tokens used for creating the decoder_input_ids from the labels
model.config.decoder_start_token_id = processor.tokenizer.cls_token_id
model.config.pad_token_id = processor.tokenizer.pad_token_id
# make sure vocab size is set correctly
model.config.vocab_size = model.config.decoder.vocab_size
# set beam search parameters
model.config.eos_token_id = processor.tokenizer.sep_token_id
model.config.max_length = 64
model.config.early_stopping = True
model.config.no_repeat_ngram_size = 3
model.config.length_penalty = 2.0
model.config.num_beams = 4
from transformers import Seq2SeqTrainer, Seq2SeqTrainingArguments
training_args = Seq2SeqTrainingArguments(
                                        predict_with_generate=True,
                                        evaluation_strategy="steps",
                                        per_device_train_batch_size=8,
                                        per_device_eval_batch_size=8,
                                        fp16=True,
                                        output_dir="./",
                                        logging_steps=2,
                                        save_steps=1000,
                                        eval_steps=100,
                                        num_train_epochs = 10
                                        )
```

11. Define the function to calculate the character error rate:

```
from datasets import load_metric
cer_metric = load_metric("cer")
def compute_metrics(pred):
    labels_ids = pred.label_ids
    pred_ids = pred.predictions
    pred_str = processor.batch_decode(pred_ids,
                                skip_special_tokens=True)
    labels_ids[labels_ids == -100] = processor.tokenizer.pad_token_id
    label_str = processor.batch_decode(labels_ids, \
                                skip_special_tokens=True)
    cer = cer_metric.compute(predictions=pred_str, references=label_str)
    return {"cer": cer}
```

12. Train the model:

```
from transformers import default_data_collator
# instantiate trainer
trainer = Seq2SeqTrainer(
                         model=model,
                         tokenizer=processor.feature_extractor,
                         args=training_args,
                         compute_metrics=compute_metrics,
                         train_dataset=train_dataset,
                         eval_dataset=eval_dataset,
                         data_collator=default_data_collator,
                         )
trainer.train()
```

The preceding code results in the following output:

Step	Training Loss	Validation Loss	Cer
100	1.468400	1.485709	0.248651
200	1.201000	1.186254	0.245412
300	1.375000	1.082842	0.106513
400	1.034500	1.100040	0.167686
500	1.183800	1.077437	0.183879
600	0.777200	0.947086	0.169845
700	0.913500	1.018770	0.154372
800	0.759700	0.909072	0.077726
900	0.965700	0.894016	0.082044
1000	0.940400	0.830890	0.065491
1100	0.811900	0.762985	0.032746
1200	0.965600	0.873897	0.087801
1300	0.729500	0.851937	0.077726
1400	0.931800	0.811440	0.066931
1500	0.759000	0.789583	0.052177
1600	0.665700	0.761649	0.064772
1700	0.625800	0.721781	0.038143
1800	0.660300	0.713733	0.041742
1900	0.695700	0.720314	0.033105
2000	0.595400	0.661299	0.017992

Figure 15.12: Training and validation loss along with the character error rate over increasing epochs

Note that the error rate kept reducing over increasing steps.

13. Perform inference on a sample image from our dataset:

```
# Load and preprocess the image
image = Image.open("/content/synthetic-data/American@3WPOqS.png").convert("RGB")
pixel_values = processor(image, return_tensors="pt").pixel_values
```

```
# Perform inference
model.eval()  # Set the model to evaluation mode
with torch.no_grad():
    generated_ids = model.generate(pixel_values.to(device))
# Decode the generated ids to text
predicted_text = processor.batch_decode(generated_ids, \
                            skip_special_tokens=True)[0]
show(image)
print("Predicted Text:", predicted_text)
```

The preceding code results in images as follows:

Figure 15.13: Sample prediction

So far, we have learned about using the encoder-decoder architecture for handwriting recognition. In the next section, we will learn about leveraging the transformer architecture to fetch keys and values within a document.

Document layout analysis

Imagine a scenario where you are tasked with extracting the values for the various keys present in a passport (like name, date of birth, issue date, and expiry date). In certain passports, values are present below the keys; in others, they are present on the right side of keys, while others have them on the left side. How do we build a single model that is able to assign a value corresponding to each text within the document image? LayoutLM comes in handy in such a scenario.

Understanding LayoutLM

LayoutLM is a pre-trained model that is trained on a huge corpus of document images. The architecture of LayoutLM is as follows:

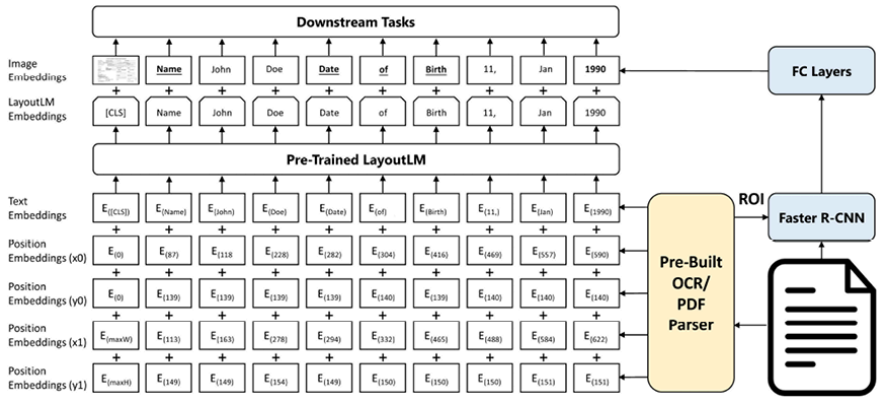

Figure 15.14: LayoutLM architecture (source: https://arxiv.org/pdf/1912.13318)

As shown in the preceding diagram, the corresponding workflow consists of the following steps:

1. We take an image of the document and extract the various words and their bounding-box coordinates (x0, x1, y0, and y1) – this is done using tools that help in OCR where they provide not only the text but also the bounding box in which the text is present in the document.
2. We take the position embeddings corresponding to these bounding-box coordinates – position embeddings are calculated based on the bounding-box coordinates.
3. We add the embeddings corresponding to the various texts extracted (**text embeddings** in the above picture) where we pass the text through a tokenizer, which in turn is passed through a pre-trained **Bi-directional Encoder Representation of Transformers** (**BERT**)-like model.
4. During the training phase of the pre-trained LayoutLM, we randomly mask certain words (but not the position embeddings of those words) and predict the masked words given the context (surrounding words and their corresponding position embeddings).
5. Once the pre-trained LayoutLM model is fine-tuned, we extract the embeddings corresponding to each word by summing up the text embeddings of the word with the position embeddings corresponding to the word.
6. Next, we leverage Faster R-CNN to obtain the image embedding corresponding to the location of the word. We leverage image embedding so that we obtain key information regarding the text style (for example, bold, italics, or underlined) that is not available with OCR.
7. Finally, we perform the downstream task of extracting the keys and values corresponding to the image. In the case of document key value extraction, it translates to the task of named entity recognition, where

each output word is classified as one of the possible keys or a value associated with a key.

LayoutLMv3 is an improvement over LayoutLM, and its architecture is as follows:

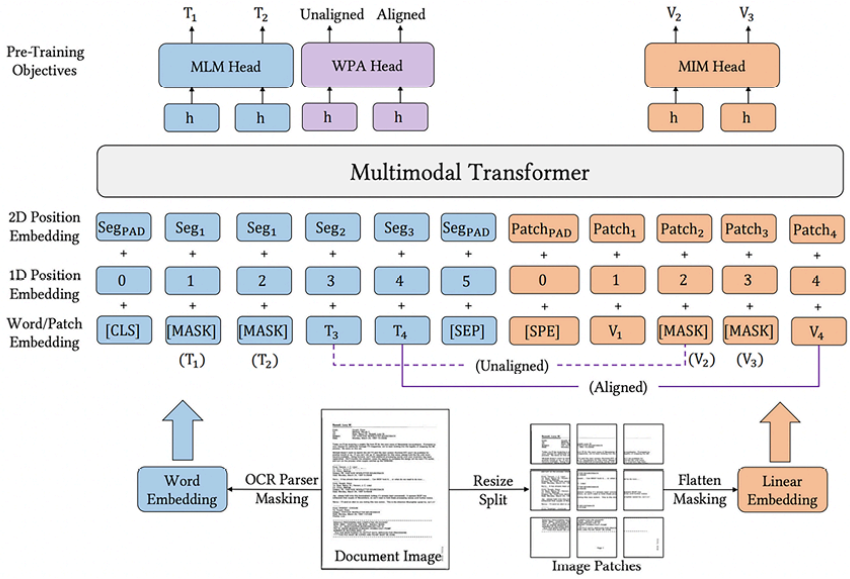

Figure 15.15: LayoutLMv3 architecture (source: https://arxiv.org/pdf/2204.08387)

The preceding architecture shows the following steps:

1. Words are obtained from an image using a typical OCR parser.
2. The words are then converted into embeddings using the `RoBERTa` model (more details here: https://arxiv.org/abs/1907.11692).
3. The document is resized into a fixed shape and then converted into multiple patches.
4. Each patch is flattened and passed through a linear layer to obtain embeddings corresponding to the patch.
5. The 1D position embeddings correspond to the index of the word/patch while the 2D position embeddings correspond to the bounding box/segment.
6. Once the embeddings are in place, we perform masked pre-training (`MLM Head`) in a manner similar to that of LayoutLM, where we mask certain words and predict them using the context. Similarly in **masked image modeling** (**MIM Head**), we mask certain blocks and predict the tokens within the block.
7. **Word patch alignment** (**WPA Head**) is then performed, which refers to the task of predicting whether a masked image patch has the corresponding tokens masked. If a token is masked and the corresponding

image patch is masked, it is aligned; it is unaligned if one of these is masked and the other isn't.

In the following section, we'll learn how to implement this.

Implementing LayoutLMv3

Let us code up `LayoutLMv3` using a passport dataset – where we try to associate each token in the image to the corresponding key or value:

The following code is available in `LayoutLMv3_passports.ipynb` in the `Chapter15` folder of this book's GitHub repository at **https://bit.ly/mcvp-2e**.

1. Install the required packages and import them:

```
%pip install transformers[torch] datasets seqeval torch-snippets torchinfo lovely_te
from torch_snippets import *
from builtins import print
```

2. Import a dataset of synthetic passports:

```
from datasets import load_dataset
dataset = load_dataset('sizhkhy/passports')
```

The input dataset contains words, boxes, and labels, as follows:

```
image <PIL.PngImagePlugin.PngImageFile image mode=RGB size=600x350 at 0x7C199E43E230>
label_string ['O', 'O', 'O', 'O', 'O', 'O', 'O', 'O', 'O', 'O', 'O', 'O', 'O', 'O', 'O'
words ['BELGIE', 'BELGIQUE', 'BELGIEN', 'BELGIUM', 'Type', '/', 'Type', 'Typ', '/', 'Ty
labels [5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5
boxes [[148, 37, 268, 80], [300, 35, 473, 80], [505, 35, 650, 80], [686, 35, 833, 77],
```

Figure 15.16: Sample expected output

A sample image is as follows:

Figure 15.17: Sample synthetic passport

3. Specify the train and test split:

```
examples_train = dataset['train']
examples_eval = dataset['valid']
```

4. Specify the `label2id` and `id2label` mapping:

```
id2label = {i:v for i, v in set(list(zip(\
                            flatten(examples_train['labels']), \
                            flatten(examples_train['label_string']))))}
label2id = {v:i for i, v in id2label.items()}
```

5. Define the processor and prepare the function to encode inputs:

```
from transformers import AutoProcessor
processor = AutoProcessor.from_pretrained("microsoft/layoutlmv3-base", apply_ocr=Fal
def prepare_examples(examples):
  images = examples['image']
  words = examples['words']
  boxes = examples['boxes']
  word_labels = examples['labels']
  encoding = processor(images, words, boxes=boxes, \
                       word_labels=word_labels,
                       truncation=True, padding="max_length")
  return encoding
```

In the preceding code, we are leveraging the pre-trained `LayoutLMv3` model's processor, which we are fine-tuning for our dataset. We are then passing the image, words, and boxes through the processor to get the corresponding encoding.

6. Prepare the train and evaluation datasets:

```
train_dataset = examples_train.map(
                              prepare_examples,
                              batched=True,
                              remove_columns=examples_train.column_names,
                              )
eval_dataset = examples_eval.map(
                              prepare_examples,
                              batched=True,
                              remove_columns=examples_eval.column_names,
                              )
```

7. Define the evaluation metric:

```
from datasets import load_metric
metric = load_metric("seqeval")
```

8. Define the function to calculate the metrics:

```
return_entity_level_metrics = False
def compute_metrics(p):
    predictions, labels = p
    predictions = np.argmax(predictions, axis=2)
    # Remove ignored index (special tokens)
    true_predictions = [[id2label[p] for (p, l) in \
                 zip(prediction, label) if l != -100] \
          for prediction, label in zip(predictions, labels)]
    true_labels = [[id2label[l] for (p, l) in \
                  zip(prediction,label) if\ l != -100] \
           for prediction, label in zip(predictions,labels)]
    results = metric.compute(predictions=true_predictions,
                                      references=true_labels)
    if return_entity_level_metrics:
        # Unpack nested dictionaries
        final_results = {}
        for key, value in results.items():
            if isinstance(value, dict):
                for n, v in value.items():
                    final_results[f"{key}_{n}"] = v
            else:
                final_results[key] = value
        return final_results
    else:
        return {
            "precision": results["overall_precision"],
            "recall": results["overall_recall"],
            "f1": results["overall_f1"],
            "accuracy": results["overall_accuracy"]}
```

In the preceding code, we are fetching the output that is not padding in ground truth and computing the precision, recall, F1 score, and accuracy metrics.

9. Define the pre-trained `LayoutLMv3` model by importing the relevant module from the transformers library:

```
from transformers import LayoutLMv3ForTokenClassification
model = LayoutLMv3ForTokenClassification.from_pretrained(
                      "microsoft/layoutlmv3-base",
                      id2label=id2label,
                      label2id=label2id
                      )
```

In the preceding code, we are defining the base model that we will use to fine-tune the model.

10. Define the training parameters:

```
from transformers import TrainingArguments, Trainer
training_args = TrainingArguments(output_dir="test",
                                  max_steps=100,
                                  per_device_train_batch_size=2,
                                  per_device_eval_batch_size=2,
                                  learning_rate=1e-5,
                                  evaluation_strategy="steps",
                                  eval_steps=50,
                                  load_best_model_at_end=True,
                                  metric_for_best_model="f1")
```

11. Initialize the trainer and train the model:

```
from transformers.data.data_collator import default_data_collator
# Initialize our Trainer
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=eval_dataset,
    tokenizer=processor,
    data_collator=default_data_collator,
    compute_metrics=compute_metrics,
)
trainer.train()
```

12. Next, we perform inference (the code for which is provided in the associated GitHub repository) to get results for an input image. A sample

inference with the predicted keys and values (present as bounding boxes within the image) is as follows:

Figure 15.18: Predicted output with bounding boxes of different keys and values extracted

In this section, we've learned about extracting keys and values from a document. In the next section, we will learn about performing question answering given a generic image.

Visual question answering

Imagine a scenario where you are given an image and are asked to answer certain questions by looking at that image. This is a task of **visual question answering** (**VQA**). A high-level strategy for VQA could be to leverage a pre-trained image to encode image information, encode the question (text) using a **large language model** (**LLM**), and then use the image and text representations to generate (decode) the answer – essentially, a multimodal model, which has input in both text and image mode.

One way of performing visual question answering is by getting the caption corresponding to the image and then performing question answering on the caption.

To understand the reason why we cannot use this, let's look at the following image:

Figure 15.19: Sample image

A set of possible questions for the same caption of the original image are:

Extracted caption	Question
A cat wearing sunglasses	What is the subject of the image?
A cat wearing sunglasses	What is the background color in the image?

Table 15.1: Some questions for the given image

In the preceding scenario, while we can answer the first question, we are unable to answer the second as the information related to the question is not extracted in the context (extracted caption).

Let's learn about BLIP2 – a model that helps in addressing this problem.

Introducing BLIP2

One of the problems associated with encoder-decoder models (as discussed in the high-level strategy we just discussed, where captioning is the process of encoding, and question answering on the caption is the process of decoding) is that the model is likely to result in **catastrophic forgetting** when VQA models integrate both visual perception and language understanding. When these models are updated with new visual or linguistic patterns, the complex interplay between these two domains can lead to the forgetting of previously learned patterns.

BLIP with frozen image encoders and LLMs (BLIP2) addresses this with a unique architecture, as follows:

Figure 15.20: BLIP2 architecture (source: https://ar5iv.labs.arxiv.org/html/2301.12597)

The **querying transformer** (**Q-Former**) acts as a bridge between the image encoder and the LLM.

The Q-Former extracts information from the image that is most relevant to the question that is asked; so, in the scenario presented at the start of this section, it would extract visual information from the image that is most relevant to the question asked. Now, we append the context (information extracted by Q-Former) to the question asked and provide it as an input to the LLM.

Essentially, Q-Former acts as a bridge between the image encoder and LLM to modify the features extracted from the image in such a way that they are most relevant to the question asked.

There are two stages in which Q-Former is trained:

1. Bootstrapping vision-language **representation learning** from a frozen encoder
2. Bootstrapping vision-language **generative learning** from a frozen LLM

Let's look at these stages in detail.

Representation learning

In the representation learning stage, we connect Q-Former to a frozen image encoder and perform pre-training using image-text pairs. We aim to train the Q-Former such that the queries (32 learnable query vectors) can learn to extract the visual representation that is most relevant to the text.

We jointly optimize three pre-training objectives that share the same input format and model parameters. Each objective employs a different attention masking strategy between queries and text to control their interaction.

The following diagram illustrates this:

Figure 15.21: Details of representation learning (source: https://arxiv.org/pdf/2301.12597.pdf)

As shown in the preceding image, the three objectives are:

- **Image-text matching:** In this task, within the self-attention module, query tokens can attend to text tokens and the text tokens can attend to query vectors. The objective of this pre-training is to perform a binary classification of whether the image and text pair match.
- **Image-grounded text generation:** In this task, within the self-attention module, query vectors do not have access (are masked) to the text tokens while the text tokens have access to the query vectors and also the previously generated tokens. The objective of this training is to generate text that matches the image.
- **Image-text contrastive learning:** In this task, we have the self-attention module that is shared between the learned queries and the input text. The learned queries interact with the image encoder to get an output vector. The input text is converted to embedding vectors and interacts with the self-attention and feed-forward network to generate an embedding corresponding to the text. We pre-train the Q-Former to have a high similarity for matching image-text pairs and a low similarity for an image and a different text (similar to how CLIP is trained). Note that while the self-attention layer is common, text tokens are masked from image tokens (learned queries) so that information does not leak between the two.

With the above three pre-training objectives, we have completed the representation learning exercise, where we extract the visual information that is most informative of the text.

Generative learning

In the generative learning stage, we pass the learned queries through the Q-Former to get an output vector, which is then passed through a fully connected layer to get an embedding that has the same dimensions as that of the text embedding dimensions. This can be illustrated as follows:

Figure 15.22: Details of generative learning (source: https://ar5iv.labs.arxiv.org/html/2301.12597)

We now have two ways of generating text through the frozen LLM. A decoder-based LLM takes the output of the **fully connected** (**FC**) layer and generates the output text, while an encoder-decoder-based model takes

the concatenation of FC output and prefix text to generate subsequent text.

With the above, we are done with the two stages of pre-training BLIP2 and getting the context that is most relevant to the question asked. This is how we provide the most relevant image context to a question and generate an answer. Let's go ahead and implement BLIP2 in the next section.

Implementing BLIP2

In this section, we will leverage BLIP2 to perform the following tasks:

- Image captioning
- Image question answering

You can use the following steps to achieve this:

This code is available in the `Visual_Question_answering.ipynb` file in the `Chapter15` folder of this book's GitHub repository at **https://bit.ly/mcvp-2e**.

1. Import the required packages and load the model:

```
import torch
from transformers import AutoProcessor, Blip2ForConditionalGeneration
device = "cuda" if torch.cuda.is_available() else "cpu"
MODEL_ID = "Salesforce/blip2-opt-2.7b"
processor = AutoProcessor.from_pretrained(MODEL_ID)
model = Blip2ForConditionalGeneration.from_pretrained(MODEL_ID,
                                torch_dtype=torch.float16)
model.to(device)
```

Note that the model requires a high VRAM and thus we have used a V100 machine on Colab.

2. Load the image – you can use any image of your choice:

```
import requests
from PIL import Image
image = Image.open('/content/Tejas.jpeg')
```

Figure 15.23: Sample image

3. Pass the image through the processor to generate a caption corresponding to the image:

```
inputs = processor(image, return_tensors="pt").to(device,
                                                  torch.float16)
generated_ids = model.generate(**inputs, max_new_tokens=20)
generated_text = processor.batch_decode(generated_ids, skip_special_tokens=True)[0].
print(generated_text)
```

This gives us the following output:

```
a baby wearing a party hat sits on a bed
```

4. Perform question answering on the image:

```
prompt = "Question: what is the color of baby's trousers? Answer:"
inputs = processor(image, text=prompt, return_tensors="pt").to(device, torch.float16
generated_ids = model.generate(**inputs, max_new_tokens=10)
generated_text = processor.batch_decode(generated_ids, skip_special_tokens=True)[0].
print(generated_text)
```

The preceding code results in the output `blue`.

Note that when performing question answering, we should explicitly mention the start of the question and the start of the answer, as provided in the prompt.

Summary

In this chapter, we learned how transformers work. Furthermore, we learned about leveraging ViTs to perform image classification. We then learned about document understanding while learning about leveraging TrOCR for handwriting transcription and LayoutLM for key-value extraction from documents. Finally, we learned about visual question answering using the pre-trained BLIP2 model.

With this, you should be comfortable in tackling some of the most common real-world use cases, such as OCR on documents, extracting key-value pairs from documents, and visual question answering on an image (handling multimodal data). Furthermore, with the understanding of transformers, you are now in a good position to dive deep into foundation models in the next chapter.

Questions

1. What are the inputs, steps for calculation, and outputs of self-attention?
2. How is an image transformed into a sequence input in a vision transformer?
3. What are the inputs to the BERT transformer in a LayoutLM model?
4. What are the three objectives of BLIP?

Learn more on Discord

Join our community's Discord space for discussions with the authors and other readers:

https://packt.link/modcv